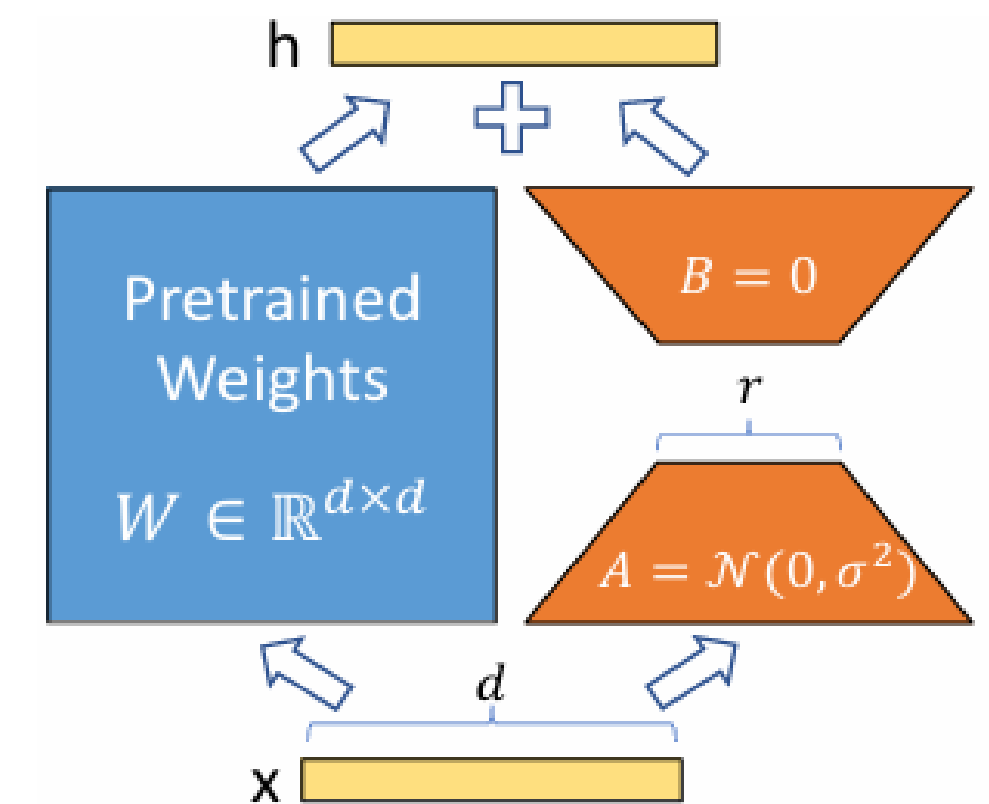


LORA: LOW-RANK ADAPTATION OF LARGE LANGUAGE MODELS

Introduction

- 대규모 일반 도메인 데이터로 사전학습을 하고, 특정 작업이나 도메인에 맞게 적응시키는 것: NLP의 중요한 패러다임 -> **But!** 모델이 점점 커질수록, 모든 파라미터를 다시 학습하는 full fine-tuning은 비현실적 (=비쌈)
Ex) 특히, GPT-3 (175B)짜리를 각 task마다 따로 저장/배포하는 것은 비용적으로도 매우 비쌈.
- Fine-tuning 방식의 큰 **단점**: 새로 만들어진 모델이 원래 모델과 동일한 수의 파라미터를 가짐.
- **기존 방법 (adapter / prefix tuning / bitfit) - 일부 파라미터만 조정/외부 모듈 학습하여 완화 -> 문제점!**
 - : inference latency 증가 (adapter)
 - : input 길이 감소 (prefix)
 - : 성능 저하
- **기존 모델 weight는 고정하고 weight 변화 ΔW 를 low-rank로 학습하는 LoRA 제안**
- **LoRA**: 파라미터 수를 10,000배 감소시키고, GPU 메모리 요구량 3배 절약
- 또한, 전체 파라미터의 gradient와 optimizer 상태를 계산할 필요가 없고, A, B 행렬만 학습
- 추론 속도에 추가 지연이 없으며 (**layer를 추가하지 않기 때문**), 다른 방식과도 쉽게 결합 가능



Problem statement

- GPT 같은 사전 학습된 자동회귀 언어모델: 문맥 x 가 주어졌을 때 목표 텍스트 y 를 생성하는 확률 모델
- 실제 다운스트림 작업(요약, MRC, NL2SQL 등) 모델을 적용할 때: 각 작업마다 문맥-정답 쌍 (x_i, y_i) 형태의 데이터 셋
- “주어진 문맥에 맞는 출력 문장을 생성하는 문제”
- Fine-tuning: 결국 사전 학습된 모델을 특정 작업용 데이터 (x, y) 에 맞춰 확률 $P(y|x)$ 를 최대로 만드는 과정
 - 요약: 기사 본문 $x \rightarrow$ 요약문 y
 - MRC: 본문+질문 $x \rightarrow$ 정답 y
 - NL2SQL: 자연어 질문 $x \rightarrow$ SQL 쿼리 y
- Full Fine-tuning: 사전 학습된 모델의 모든 파라미터 W_0 를 시작점으로 두고, 각 작업 데이터 (x, y) 에 대해 조건부 확률 최대화하도록 전체 가중치 $W_0 + \Delta W$ 를 업데이트하는 방식 $\rightarrow W_0$ 를 전부 업데이트
 $\rightarrow$ 저장/배포/관리 비용 증가
- LoRA approach: 기존 가중치 W_0 freeze, 변화량만 따로 아주 작은 파라미터 θ 로 표현해 학습
사전 학습된 가중치 W_0 는 고정하고, $\Delta W = BA$ (low-rank)로 표현하여 학습

Limitations of Existing Methods

- Adapter layers

: Transformer 블록 사이에 새로운 작은 Layer(어댑터)를 끼워 넣는 방식

장점: 파라미터 추가량 매우 적음

단점:

- inference latency(추론 속도) 증가
- 순차적 처리 -> 온라인 환경(batch=1)에서 느림

Batch Size	32	16	1
Sequence Length	512	256	128
$ \Theta $	0.5M	11M	11M
Fine-Tune/LoRA	1449.4±0.8	338.0±0.6	19.8±2.7
Adapter ^L	1482.0±1.0 (+2.2%)	354.8±0.5 (+5.0%)	23.9±2.1 (+20.7%)
Adapter ^H	1492.2±1.0 (+3.0%)	366.3±0.5 (+8.4%)	25.8±2.2 (+30.3%)

Table 1: Inference latency of a single forward pass in GPT-2 medium measured in milliseconds, averaged over 100 trials. We use an NVIDIA Quadro RTX8000. “ $|\Theta|$ ” denotes the number of trainable parameters in adapter layers. Adapter^L and Adapter^H are two variants of adapter tuning, which we describe in Section 5.1. The inference latency introduced by adapter layers can be significant in an online, short-sequence-length scenario. See the full study in Appendix B.

- Prefix tuning

: 입력 토큰 앞에 학습 가능한 prefix 토큰을 붙여서 모델을 조정하는 방식

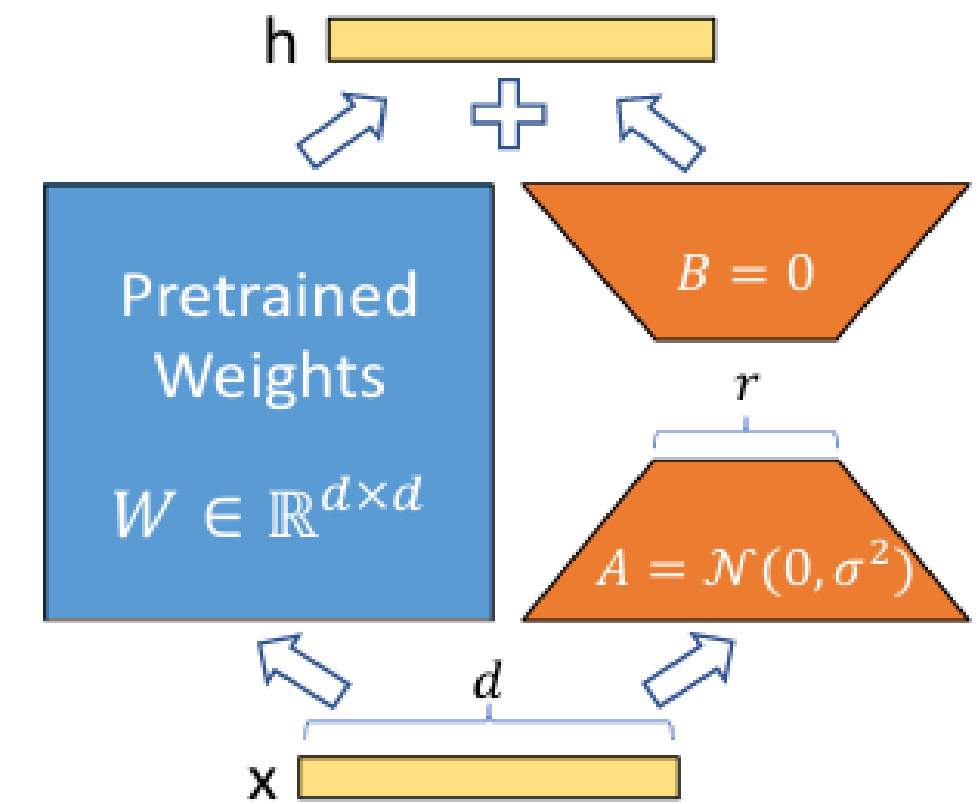
단점:

- Prompt 최적화 어려움
- 성능이 불안정함 (non-monotonic)
- 사용 가능한 문맥 길이 감소

* 기존 방법은 latency 또는 성능 문제 존재 ⇒ LoRA 필요

Low-Rank-Parametrized update matrices

- Transformer의 dense layer 가중치 W_0 는 보통 full rank.
- 연구에 따르면 사전학습된 모델은 낮은 차원의 구조를 가짐
→ “가중치 변화도 실제로는 저랭크로 표현 가능하다” 가정
- LoRA 아이디어: 기존 업데이트를 대형 행렬로 직접 학습하는게 아니라, 작은 두 행렬 BA로 분해해서 학습하자.
원래 업데이트 방식: $W_0 + \Delta W$
LoRA 방식: $\Delta W = B * A$ (ΔW : Adaptation 중에 누적된 기울기 업데이트)
- W_0 = 사전학습된 본체 모델 / BA = 작업별 추가 보정 패치 / B: ($d \times r$), A: ($r \times k$), r: LoRA 모듈의 rank
- 훈련방식: W_0 는 freeze, A, B만 학습
- 새로운 forward pass: $h = W_0 x + B A x$ / 원래 출력에 LoRA가 만든 작은 변화량을 더해주는 방식
- 초기값 → A=랜덤 가우시안 초기화 / B=0으로 초기화



왜 B=0?

→ B=0 이면 BA=0 → LoRA 영향 '0' → 안전한 시작 → 그 다음 학습이 진행되면 B가 업데이트 되면서 점점 LoRA 변화 시작

APPLYING LORA TO TRANSFORMER

- Transformer= Attention + MLP -> LoRA: Attention만 적용 (MLP freeze)
- 즉, query, key, value, output projection 행렬인 W_q, W_k, W_v, W_o 중 W_q 와 W_v 만을 적응
- Why?
 - : Attention이 실제 성능에 더 큰 영향
 - : MLP까지 LoRA 넣으면 파라미터가 너무 많아짐
 - : 복잡도 증가 -> 연구 초점 분명히 하려고 Attention만 실험함
- 효과
 - : VRAM 사용량 2~3배 감소
 - : GPT-3 175B도 1.2TB -> 350GB 감소
 - : 체크포인트 크기 10,000배 줄어듦
 - : 작업 전환 시 BA만 교체하면 돼서 비용 거의 0
 - : 학습 속도 25% 향상
- 한계
 - : 여러 작업을 한 배치에 섞어 처리하기 어려움
- 성능
 - : No additional inference latency
 - : Inference 시 $W_0 + BA$ 로 merge

Baselines

LoRA -> 기존 다양한 parameter-efficient 방법들과 비교

대표적으로 adapter, prefix tuning, bitfit 등이 있으며, 각각 다른 방식으로 파라미터 줄임,
LoRA는 weight 업데이트를 low-rank로 표현하는 방식

- Full Fine-tuning(FT) - 모든 파라미터 업데이트
- BitFit – bias만 학습
- Prefix tuning – 입력에 trainable token 추가
- Prefix-layer tuning – 각 layer의 activation 직접 학습
- Adapter – 중간에 작은 layer 추가
- LoRA – $\Delta W = BA$ (low-rank), $W_q \cdot W_v$ 에 적용